

Overview

Our distributed task runner implements a Master-Worker pattern using MPI for interprocess communication. The system consists of one master process (rank 0) that coordinates task distribution and multiple worker processes that execute tasks. The tasks are managed in a queue and are processed on a first-in-first-out (FIFO) basis.

3 message types implemented via MPI tags:

1. **TAG_NEW_TASK** (1): Master assigning a task to a worker
2. **TAG_NEW_RESULT** (2): Worker returning execution results and child tasks
3. **TAG_TERMINATE** (3): Master signalling a worker to terminate

Work Distribution Algorithm

Master Process:

1. Read initial tasks from input file
2. Distribute one task to each available worker
3. Keep track of idle workers in the `idle_workers` vector
4. Receive results and new tasks from worker and add worker to idle workers pool
5. Assign new tasks to idle workers
6. Terminate all workers once all tasks are completed (i.e. no tasks remain in queue) so all workers are idle (`while (static_cast<int>(idle_workers.size()) < num_procs - 1)`)

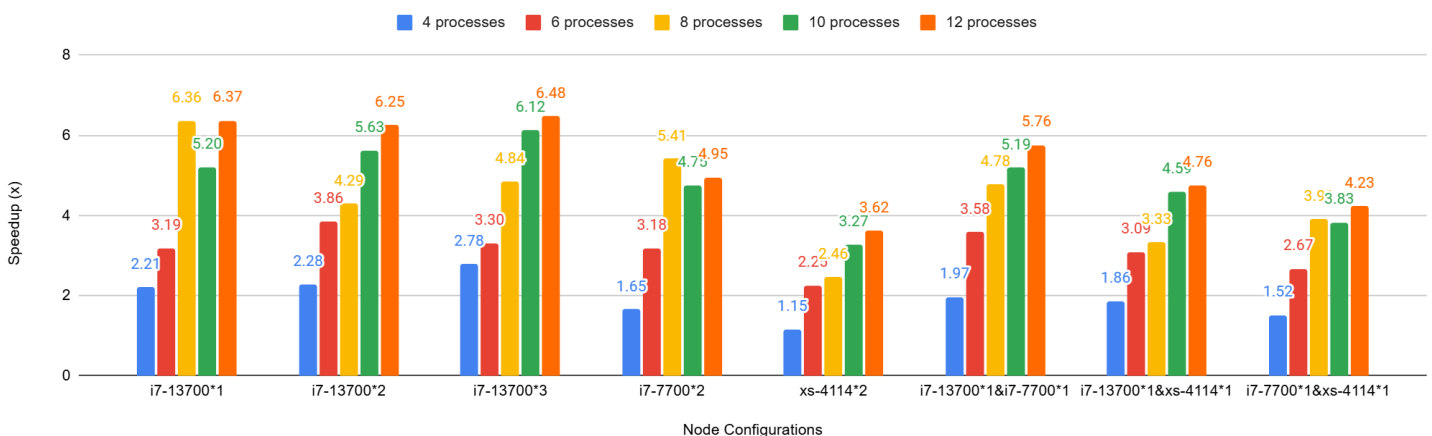
Worker Process:

1. Receive task from master
2. Execute task, potentially generating child tasks
3. Send number of child tasks and task data back to master
4. Repeat until termination signal is received from master

Performance Analysis

Node Configurations

Effect of Node Configurations on Speedup



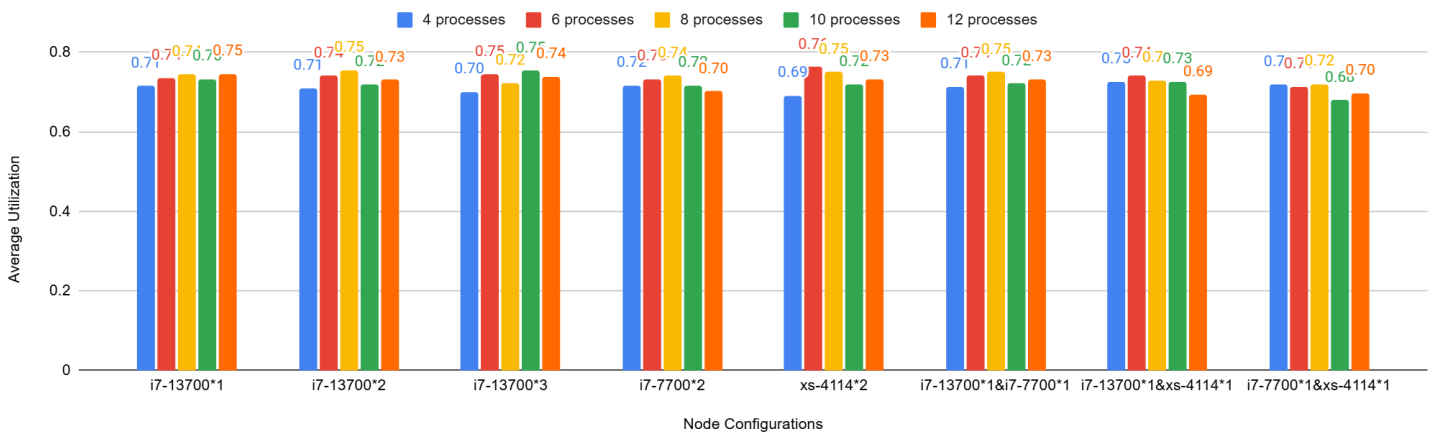
As the number of processes increases, the runtime and speedup increases roughly linearly since the workload can be distributed across more processes. The increase is linear following Gustafson's Law since the problem size is sufficiently large to keep all processors busy. There are minimal sequential bottlenecks since the sequential portion of the program managed by the master process remains constant as processes increase and task management operations are lightweight

compared to task computation. The parallelisable portion can also scale linearly with added processes since each task can run independently with communication only at task boundaries and there are no blocking dependencies, such as global barriers or collective operations.

From the speedups, running on i7-13700 nodes gives the greatest speedup, followed by i7-7700, then xs-4114, suggesting i7-13700 nodes are the fastest, followed by i7-7700, then xs-4114, which corresponds to their respective clock speeds.

The runtime and speedup for mixed configurations with a combination of node types falls between the homogeneous configurations with the same number of each node type as tasks on faster nodes complete quicker while tasks on slower nodes take longer and the final runtime depends on the last completed task. The runtime and speedup is above the average of the homogeneous configurations likely due to load balancing challenges (since the master process cannot perfectly predict task execution times on heterogeneous hardware) and inter-node communication between different CPU architectures may introduce additional overhead.

Effect of Node Configurations on Average Utilization



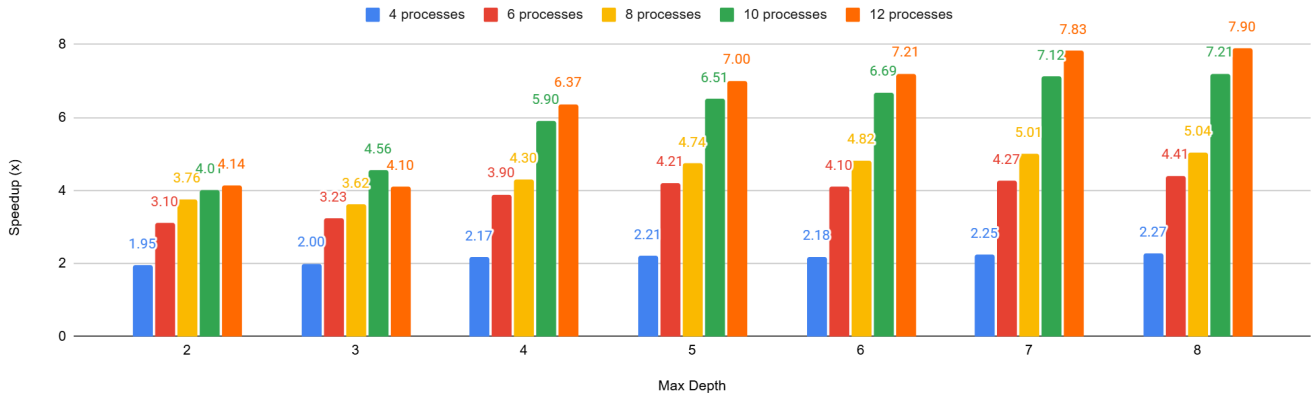
Average utilization remains roughly constant across all node configurations and any number of processes. As our implementation uses a master-worker pattern, it leads to a consistent ratio between computation and communication and hence utilization, since all task assignments flow through the single master process, creating a consistent communication bottleneck.

Maximum depth of tasks generated in the task graph

There is a slight increase in speedup as the maximum task tree depth increases, due to more fine-grained parallelism and better task distribution across processes. However, the gains are modest because deeper task trees also introduce overhead from task management and communication. This leads to diminishing returns when the task granularity surpasses what the hardware can efficiently handle. The increased concurrency helps up to a point, but without enough cores or efficient scheduling, the additional depth can lead to performance saturation.

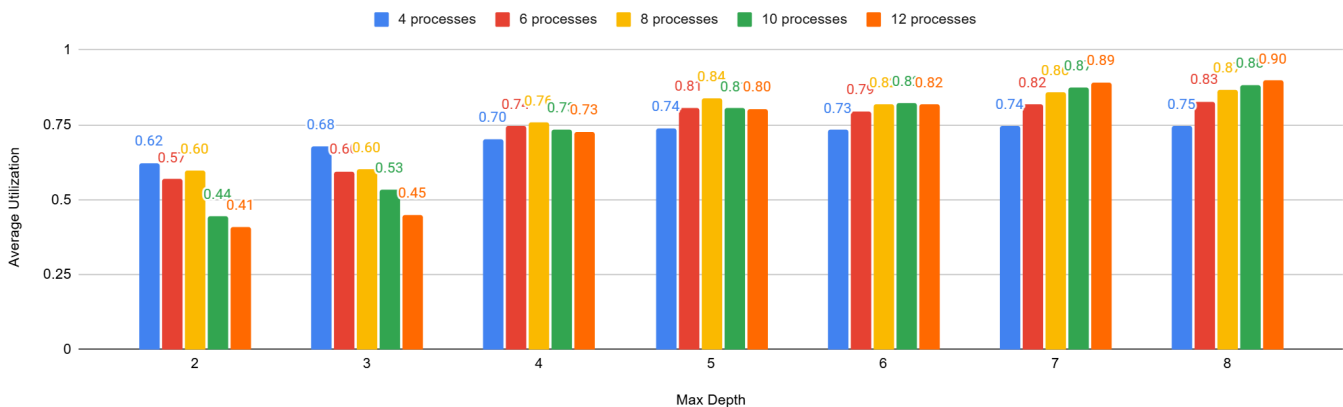
Scaling MPI processes on the i7-13700 reveals chunked performance improvements, with notable gains from 4 to 6 and 8 to 10 processes, but smaller improvements between 6 to 8 and 10 to 12. This is due to the CPU's hybrid architecture: 8 Performance (P) cores and 8 Efficient (E) cores, and 20 threads. Initial gains reflect use of high-performance P-cores, while later gains involve E-cores or hyper-threading, which provide less benefit. As more processes compete for limited physical resources, cache contention and scheduling overhead reduce the effectiveness of further parallelism.

Effect of Max Depth on Speedup



Average utilisation increases as the maximum task depth grows, since more tasks are continuously available to keep worker processes busy. At lower depths, this trend isn't clear due to the overhead of spawning and managing tasks resulting in worker processes waiting for tasks, which can outweigh the benefits of parallelism. As the depth increases, task granularity improves, and the system approaches ideal utilisation levels, with worker processes staying active for longer periods.

Effect of Max Depth on Average Utilization



In a master-worker model, the master process typically coordinates tasks and does little computation, resulting in near-zero utilisation for the master. This leads to an overall utilisation of roughly $(N-1)/N$, where N is the number of MPI processes. For example, the 12 process average utilisation is at 0.90 ($11/12 = 0.916$), with this discrepancy likely due to communication overheads. As such, one possible optimization would be to assign some computational work to the master while it waits for worker results, helping improve total resource usage and system efficiency.

Optimizations

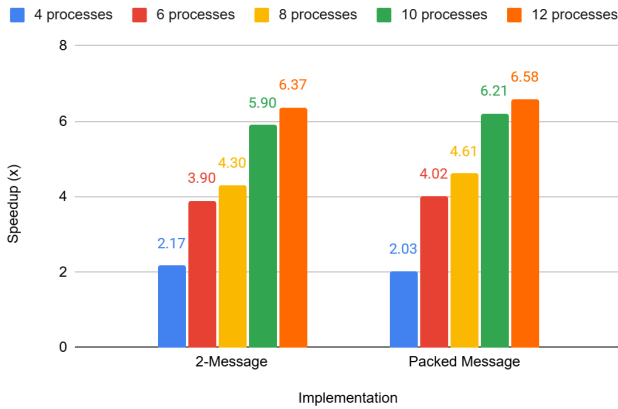
Packing of Data

`MPI_Pack` and `MPI_Unpack` are used to bundle the number of generated tasks and the tasks themselves into a single message when workers send results back to the master. This reduces the number of individual MPI messages, making the communication process more efficient and less prone to synchronization delays between sending and receiving. It simplifies message handling on both ends and can reduce message latency in scenarios where communication overhead is non-negligible.

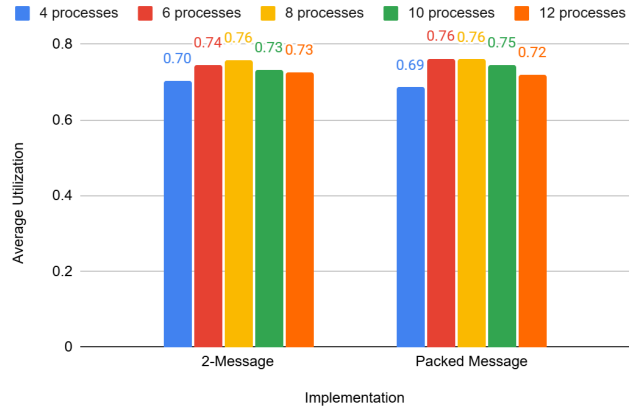
However, in this application, the dominant cost is in task execution rather than communication, as evidenced from the high average utilization. The master's role in dispatching tasks and receiving

results is lightweight, and communication occurs infrequently relative to computation time. This is reinforced by the master's CPU utilization being zero. As such, even though message packing reduces communication complexity, it only results in a marginal increase in overall speedup and CPU utilisation. The system remains compute-bound, and the benefits of communication optimizations are minimal under such conditions.

Effect of Packing Optimization on Speedup



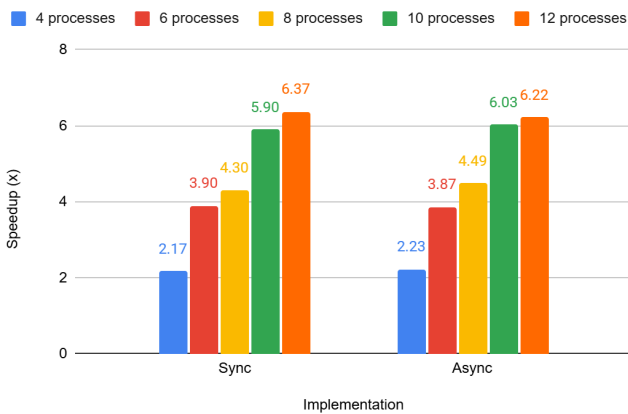
Effect of Packing Optimization on Average Utilization



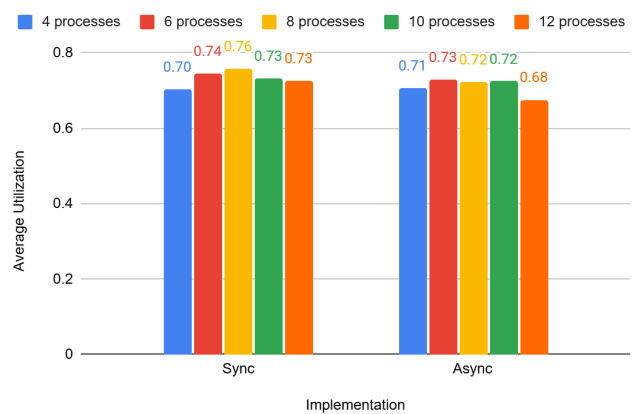
Non-blocking Communications

In our implementation of asynchronous sends, the master asynchronously dispatches tasks to workers, and workers are able to receive and process tasks concurrently. However, due to pointer overriding and message handling constraints, both the master and workers must perform synchronization at certain points: the master needs to ensure that both the number of new tasks and task results are received before dispatching tasks, and workers must receive the task data before processing it. These necessary waits introduce synchronization delays that limit the potential benefits of asynchronicity.

Effect of Async Optimization on Speedup



Effect of Async Optimization on Average Utilization



Since the master's primary role is dispatching, which is not computationally intensive, its CPU utilisation remains near zero even with async sends. As a result, the communication overhead is not effectively hidden, and the expected overlap between communication and computation does not materialize. This explains why the observed results show insignificant improvements in speedup and utilisation. While async sends offer theoretical performance gains, their effectiveness is bottlenecked by coordination overhead and the lightweight nature of the master's role in this workload.

Acknowledgements

ChatGPT and GitHub Copilot were used to assist in generating bash/python scripts for ease of testing and data extractions.

Appendix

Appendix A: Performance Analysis

The spreadsheet with the processed data can be found [here](#).

Variables:

Measurements for the following variables were taken. The ranges for the variables measured are given below (If the variables are not being tested, they are kept at the constant value specified unless otherwise mentioned):

Testing for each of the variables were run on 4, 6, 8, 10 and 12 processes. 3 measurements each were taken to obtain an average value.

- Node Configurations (Constant: i7-13700*2)
 - i7-13700*1, i7-13700*2, i7-13700*3, i7-7700*2, xs-4114*2, i7-13700*1&i7-7700*1, i7-13700*1&xs-4114*1, i7-7700*1&xs-4114*1
- Maximum depth of tasks (H) generated in the task graph: [4, 8] (Constant: 4)
 - 2, 3, 4, 5, 6, 7, 8
- $N_{\min} = N_{\max} = 2$
- $P = 0.00$ (Negligible when $N_{\min} = N_{\max}$)

Appendix B: Raw Data

Results from varying Node Configurations

Number of Processes			4			6			8			10			12		
	Sequential		Average			Average			Average			Average			Average		
Node Configurations	Runtime (ms)	Speedup (x)	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization
i7-13700*1	48,568	1.0000	22,020	2.205631 24432334	0.71490	15,230	3.188969 13985555	0.73614	7,631	6.364287 58626714	0.74480	9,340	5.1998144248 9561	0.73370	7,623	6.37152352 632500	0.74645
i7-13700*2	48,568	1.0000	21,302	2.280009 38893670	0.71016	12,588	3.858175 55938038	0.74158	11,327	4.287681 71384851	0.75477	8,633	5.62607151131 362	0.71926	7,768	6.25258550 401236	0.73094
i7-13700*3	48,568	1.0000	17,441	2.784649 49162908	0.70060	14,711	3.301475 08666984	0.74503	10,037	4.838735 38788523	0.72211	7,938	6.11841773746 536	0.75397	7,499	6.47630900 524491	0.73830
i7-7700*2	48,568	1.0000	29,401	1.651916 60147614	0.71770	15,291	3.176178 22731831	0.73339	8,976	5.4110743 8630371	0.74123	10,219	4.7525605062 3002	0.71772	9,803	4.95440171 376109	0.70205
xs-4114*2	48,568	1.0000	42,168	1.151764 75238133	0.69194	21,576	2.251019 65146459	0.76296	19,771	2.456568 65389803	0.75009	14,870	3.2661735036 9872	0.71938	13,434	3.61539415 895387	0.73111
i7-13700*1 &i7-7700*1	48,568	1.0000	24,715	1.965122 39530649	0.71396	13,552	3.583737 11783949	0.74185	10,164	4.778276 98160233	0.75007	9,359	5.1892584941 9474	0.72168	8,429	5.76201210 107961	0.73105
i7-13700*1 &xs-4114*1	48,568	1.0000	26,095	1.861199 46349875	0.72655	15,703	3.092846 52939928	0.74184	14,586	3.329692 17760918	0.73022	10,576	4.5924291612 8219	0.72722	10,200	4.76172423 935423	0.69328
i7-7700*1& xs-4114*1	48,568	1.0000	32,055	1.515161 59893515	0.71986	18,218	2.665886 01225871	0.71376	12,432	3.906797 15779595	0.72067	12,681	3.82988119020 082	0.67954	11,487	4.22820661 636680	0.69701

Results from varying Maximum Depth of tasks (H) generated in the task graph

Number of Processes			4			6			8			10			12		
	Sequential		Average			Average			Average			Average			Average		
Max Depth	Runtime (ms)	Speedup (x)	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization
2	11,140	1.0000	5,727	1.9450587 8244675	0.62298	3,592	3.1016241 2993039	0.56803	2,963	3.7592800 8998875	0.59731	2,777	4.0110417 6668267	0.44430	2,693	4.1371626 6402575	0.40999
3	26,620	1.0000	13,312	1.9996995 1923077	0.67675	8,235	3.2326748 7046632	0.59539	7,353	3.6202910 3767170	0.60011	5,844	4.5550992 4709103	0.53379	6,485	4.1046463 8157895	0.44919
4	48,568	1.0000	22,381	2.1700549 5732988	0.70345	12,456	3.8991650 6101477	0.74479	11,297	4.2993213 3372676	0.75749	8,230	5.9010975 6591471	0.73324	7,624	6.3706877 7053911	0.72654
5	77,974	1.0000	35,303	2.2086866 2071570	0.73798	18,509	4.2127613 5933870	0.80631	16,453	4.7391964 9911870	0.83836	11,975	6.5115800 0222692	0.80529	11,133	7.0038623 9108955	0.80321
6	146,938	1.0000	67,455	2.1783007 0268721	0.73285	35,812	4.1029998 9761442	0.79446	30,479	4.8209586 9287050	0.81864	21,952	6.6937058 6895452	0.82206	20,391	7.2059044 6921895	0.81846
7	264,239	1.0000	117,506	2.2487213 4551613	0.74483	61,897	4.2689884 3245805	0.81779	52,771	5.0072767 2395824	0.85879	37,089	7.1243933 5658051	0.87474	33,756	7.8279120 7489039	0.89158
8	526,072	1.0000	231,432	2.2731202 1547037	0.74605	119,269	4.4108024 7172358	0.82827	104,367	5.0405810 2471399	0.86855	72,991	7.2073214 1715455	0.88391	66,584	7.9008375 3435493	0.89720

Results from Packing Optimization

Number of Processes			4			6			8			10			12		
	Sequential		Average			Average			Average			Average			Average		
Implementation	Runtime (ms)	Speedup (x)	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization
2-Message	48,568	1.0000	22,381	2.17005495732988	0.70345	12,456	3.89916506101477	0.74479	11,297	4.29932133372676	0.75749	8,230	5.90109756591471	0.73324	7,624	6.37068777053911	0.72654
Packed Message	48,568	1.0000	23,924	2.03009530178900	0.68652	12,094	4.01576495879613	0.76265	10,530	4.61219967712323	0.76053	7,817	6.21312523986184	0.74547	7,380	6.58132706987669	0.71956

Results from Asynchronous Send & Receive Optimization

Number of Processes			4			6			8			10			12		
	Sequential		Average			Average			Average			Average			Average		
Implementatio n	Runtime (ms)	Speedup (x)	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization	Runtime (ms)	Speedup (x)	Average Utilization
Sync	48,568	1.0000	22,381	2.17005495732988	0.70345	12,456	3.89916506101477	0.74479	11,297	4.29932133372676	0.75749	8,230	5.90109756591471	0.73324	7,624	6.37068777053911	0.72654
Async	48,568	1.0000	21,825	2.22533791523482	0.70627	12,561	3.86646852775714	0.72916	10,828	4.48540820096047	0.72367	8,057	6.02829954489036	0.72419	7,810	6.21895940927910	0.67559